Now, summarise the document titled 'Blockchain in Finance'."

This model uses one example to understand the format and tone expected in the summary. One-shot prompting is beneficial for slightly more complex tasks where a single example can significantly enhance the model's performance.

Few-shot prompting

Few-shot prompting involves providing the model with several examples to help it understand the task more comprehensively. This approach is particularly effective for complex tasks. The model is given a few examples (typically 2–5) to help it learn the pattern or style before performing the task.

Example: Bank assistance

Prompt: *"Here are some examples of how to respond to customer queries: Q: How do I reset my online banking password? A: You can reset your password by clicking 'Forgot Password' on the login page and following the instructions. Q: What is the interest rate for savings accounts? A: Our standard savings account offers an interest rate of 3.5% per annum. Now, answer this: What are the charges for international wire transfers?"*

Few-shot prompting is ideal for tasks that require more context or when the task is outside the model's typical training data. With these examples, the model understands the tone, structure, and level of detail expected in banking responses.

Chain-of-thought prompting

Chain-of-thought prompting encourages the model to generate intermediate steps or reasoning before arriving at the final answer. This method is useful for tasks that involve logical reasoning or complex problem-solving. The model is encouraged to reason step-by-step before arriving at an answer.

Example: Solving a maths problem

Prompt: *"Solve the maths problem: What is 15 + 8? Let's break it down. First, add 10 to 15 to get 25.*

Model response may be*: "15 + 8 is 23. Breaking it down: 15 + 10 = 25, then 25 - 2 = 23."*

Chain-of-thought prompting is effective for enhancing the model's reasoning capabilities and ensuring that the final answer is well-justified. Here is another example.

Example: Learning a topic (e.g., quantum computing)

Prompt: *"Explain quantum entanglement to a beginner. Think step-by-step."*

The model may respond like this: *"First, let's understand what quantum particles are. These are tiny units like electrons or photons. Next, entanglement means that two particles become linked in such a way that the state of one instantly affects the other, no matter how far apart they are. This phenomenon defies classical physics and is a key concept in quantum computing."*

By reasoning through the explanation step-by-step, the model helps the learner grasp a complex topic more intuitively.

The role of tokens in prompt engineering

Tokens in AI models are the basic units that the model thinks with and uses to comprehend and process text. Mastering their role is essential for optimising efficiency, controlling costs, and ensuring precise, high-quality outputs within the model's context limits.

Tokens are not necessarily full words, depending on the model's tokenizer (e.g., GPT models use Byte Pair Encoding for tokenizing).

A token can be one of the following:

- A full word (e.g., 'hello')
- A subword (e.g., 'un' + 'believ' + 'able' for 'unbelievable')
- Punctuation, white space, or other special characters.

Tokens are the way the model 'sees' your prompt. Large models (GPT-4 and others) have a context window (e.g., 128k tokens) that restricts the total input plus output length. The information is truncated or errors can occur when the data exceeds this limit.

Here is why tokenisation is important in prompt engineering.

Efficient and cost-effective*:* Every token costs money (e.g., in terms of API transactions) and uses CPU or memory. A well-designed prompt has four token counts and increases output quality, saving costs and reducing latency.

Optimisation of the context window*:* Prompts must fit the model's token limit. Engineers use techniques such as summarisation or chain-of-thought to fit more value into fewer tokens and to make sure the model retains the critical context without flooding it.

Precision and performance*:* Tokenization influences the encoding process. Bad token usage (redundant phrasing, etc) can distort the intent of the prompt and cause hallucinations or garbage in general. On the other hand, token-aware prompts yield outputs that are clearer, more relevant, and more controllable.

Scalability for advanced techniques*:* In approaches such as few-shot learning or retrieval-augmented generation (RAG), the number of tokens decides how many examples or external data points you can add before the model starts complaining.

Debugging and iteration*:* It is useful to keep track of the tokens that aid in diagnosing problems such as why a prompt fails in generating a coherent prediction. Prompts that are too long can get models to 'forget' context.

Here's an example of the use of tokens in prompt engineering.

- ***Scenario****:* The article is about climate change (hypothetical 500-word text). We want a 100-word summary.